

Universidad del Rosario

FACULTAD DE ECONOMÍA

INGENIERÍA DE DATOS

Profesor

Tatiana Cabrera Vargas

SOLACTIVE PRICE VALIDATION

Juan Mario Delgado Molina

Nicolle Stefany Garaviz Sánchez

Joan Sebastián Schick Reyes

Noviembre 2025

Índice

1. Contextualización	5
2. Formulación del problema	5
2.1. Problema central	5
2.2. Relevancia técnica	5
2.3. Relevancia social/empresarial	5
3. Objetivos del Proyecto	6
4. Alcance	6
5. Metodología de desarrollo	7
6. Requisitos	8
6.1. Requisitos funcionales de alto nivel	8
6.2. Requisitos funcionales de bajo nivel	8
6.3. Requisitos no funcionales	15
7. Historias de Usuario para RQF seleccionados	17
8. Flujo de solución	37
8.1. Opción 1	37
8.2. Opción 2	38
8.3. Solución seleccionada	38
9. Diagrama de clases	39
10. Modelo Entidad-Relación (MER)	39
11. Modelo físico	40
12. Diccionario de datos	40
13. Ejecución de Requisitos	44
13.1. RQF 1 - Insertar Divisiones	44
13.2. RQF 2 - Consultar Divisiones	44
13.3. RQF 3 - Actualizar División	44
13.4. RQF 4 - Eliminar División	44
13.5. RQF 5 - Insertar Usuario	44
13.6. RQF 6 - Buscar Usuario por Email	44
13.7. RQF 7 - Actualizar Usuario	44

13.8. RQF 8 - Eliminar Usuario	45
13.9. RQF 9 - Insertar Resultado	45
13.10RQF 10 - Consultar Resultados	45
13.11RQF 11 - Consultar Vendors Ordenados	45
13.12RQF 12 - Eliminar Resultado	45
13.13RQF 13 - Insertar Activo	45
13.14RQF 14 - Consultar Activo por ID	45
13.15RQF 15 - Actualizar Ticker	46
13.16RQF 16 - Eliminar Activo	46
13.17RQF 17 - Insertar Vendor	46
13.18RQF 18 - Consultar Vendor por ID	46
13.19RQF 19 - Actualizar Vendor	46
13.20RQF 20 - Eliminar Vendor	46
14.Requisitos de Procedimientos Almacenados	46
14.1. RQF 20 - Filtro de Activos	46
14.2. RQF 21 - Consulta Activos por Vendor	47
14.3. RQF 22 - Consulta Vendors Ordenados	47
14.4. RQF 23 - Comparación de Precios	47
14.5. RQF 24 - Validación Manual	48
14.6. RQF 25 - Conteo de Validaciones	48
14.7. RQF 26 - Trigger de Validación Automática	49
14.8. RQF 27 - Consulta Resultados No Validados	49
14.9. RQF 28 - Consulta Inconsistencias por Vendor	49
14.10RQF 29 - Filtro Avanzado de Activos	49
14.11RQF 30 - Vista de Monitoreo de Inconsistencias	50
14.12RQF 32 - Trigger de Reversión Manual	50
14.13RQF 33 - Vista Final de Datos	51
14.14RQF 34 - Estadísticas de Validación	51
14.15RQF 35 - Consulta Activos por Vendor	52
14.16RQF 36 - Consulta Activos por Tipo	52
14.17RQF 37 - Consulta Activos por Timestamp	52
14.18RQF 39 - Modificación de Validación	53
15.Requisitos Adicionales Identificados	53
15.1. Trigger de Error de Precio	53
15.2. Vista de Validación por Ticker	54
16.Arquitectura del Proyecto	54

17. Configuración del Servidor	55
17.1. server.js	55
18. Base de Datos	56
18.1. database.js	56
19. Middlewares	56
19.1. errorHandler.js	56
19.2. validateId.js	57
20. Modelo de Datos	58
20.1. Esquema Solactive	58
21. Métodos del Modelo	59
21.1. NewSec	59
21.2. UpdateSec	60
21.3. DeleteFields	60
21.4. Validation	60
21.5. UserValidation	60
21.6. UserValidationReversal	60
21.7. ValidationCount	61
21.8. ErrorConsulting	61
21.9. ConsultingBy	61
21.10. ViewAllSecurities	61
21.11. ViewValidationCount	61
21.12. FilteredSecuritiesView	61
21.13. ErrorTypeCount	61
21.14. VersionComp	62
21.15. DeleteDocument	62
22. Controladores	62
22.1. Estructura de Respuesta	62
22.2. Endpoints Implementados	62
22.2.1. Gestión de Valores	62
22.2.2. Validaciones	62
22.2.3. Consultas	63
22.2.4. Vistas	63
22.2.5. Reportes	63
23. Rutas	63

24. Validaciones	63
25. Manejo de Errores	63
26. Configuración y Variables de Entorno	64
27. Certificaciones MongoDB	64

1. Contextualización

Solactive es un proveedor de índices con sede en Alemania que opera a nivel mundial y crece a un ritmo acelerado. Desde 2007, desarrolla soluciones de índices a medida y multiclase de activos para ETF y otros productos de inversiones indexados, con los principales bancos de inversión y gestores de activos mundiales como clientes.

La serie de índices de referencia global solactive ofrece un marco integral de índices que abarcan el mercado bursátil global. Estos índices pueden utilizarse directamente en un producto, como referencia o como universo de partida para un índice desarrollado.

2. Formulación del problema

2.1. Problema central

El segmento de Price Data de Solactive enfrenta limitaciones en la validación de precios de cierre de activos financieros. El proceso actual exige coincidencia exacta entre los tres proveedores de datos (ICE como primario; FactSet y EDI como secundarios) para clasificar un precio como `VALIDATED`; si solo coincide en dos, se marca `SEMI_VALIDATED` y, si no coincide, `UNVALIDATED`. Esta rigidez produce “falsos no válidos”: diferencias menores en unidades (p. ej., dólares vs. centavos), formatos o decimales insignificantes provocan que precios objetivamente correctos se clasifiquen como `UNVALIDATED`, interrumpiendo el flujo hacia el cálculo de índices.

2.2. Relevancia técnica

La ausencia de reglas de validación flexibles (normalización de unidades, tolerancias numéricas, redondeos consistentes y armonización de formatos) obliga a recalcular índices de forma manual para corregir discrepancias triviales. Esto aumenta tiempos de procesamiento, eleva la carga operativa y reduce la confiabilidad del algoritmo de validación. Un sistema con reglas parametrizables y pre-procesamiento estandarizado mejoraría la calidad de datos de entrada, permitiría trazabilidad/auditabilidad de las decisiones de validación y reduciría retrabajos, con impacto directo en el rendimiento del índice.

2.3. Relevancia social/empresarial

Los retrasos y retrabajos repercuten en los plazos de entrega comprometidos con clientes institucionales, afectando la satisfacción, la confianza y la percepción de calidad del servicio. Minimizar los falsos `UNVALIDATED` y automatizar correcciones reduce costos operativos, estabiliza los tiempos de publicación de índices y disminuye incidentes con contrapartes que emiten productos financieros basados en dichos índices. En suma,

abordar el problema preserva la reputación de Solactive, mejora la experiencia de vendors y clientes y fortalece la competitividad en un mercado donde la precisión y la puntualidad son críticas. Si el precio coincide en los tres vendors, se considera VALIDATED.

3. Objetivos del Proyecto

Desarrollar una solución tecnológica soportada en una base de datos relacional y no relacional que facilite la validación, carga y depuración de los datos de precios de activos provenientes de distintos vendors.

- Establecer un conjunto de reglas para el tratamiento de los datos invalidados
- Reducir la necesidad de cálculos manuales
- Garantizar la consistencia de la información enviada a los clientes

4. Alcance

- Delimitación funcional:
 - Se propone automatizar la comparación de la información del cierre de mercado proporcionada por los tres proveedores: ICE, FactSet y EDI.
 - La herramienta verificará que los datos sean idénticos en al menos dos de los tres proveedores.
- Delimitación técnica:
 - La limpieza y transformación de los datos se realizarán utilizando Visual Studio Code como entorno de desarrollo.
 - El proyecto se implementará en Python, empleando librerías como Pandas y NumPy para la manipulación y análisis de la información.
 - La base de datos relacional se empleará para almacenar los registros procesados y mantener la trazabilidad de los resultados.

- Esta configuración técnica permitirá un manejo automatizado, eficiente y confiable de los datos.
- Alcances excluidos:
 - No contempla la integración directa con plataformas de información financiera ni con sistemas en tiempo real.
 - No se desarrollará una interfaz gráfica avanzada, ya que el objetivo principal es la automatización interna de los procesos, no el diseño visual.
 - No se incluirán funciones adicionales distintas a la comparación de los datos de cierre.
- Alcances futuros:
 - Se proyecta la integración del sistema con servicios en la nube, con el fin de mejorar su escalabilidad, rendimiento y disponibilidad.

5. Metodología de desarrollo

Hemos creado un tablero Kanban en Trello que demuestra nuestro trabajo como repartición de tareas. El tablero está dividido en las siguientes columnas o listas:

- **Backlog:** Esta es nuestra lista de tareas pendientes. Aquí se encuentran todas las ideas y funcionalidades que queremos implementar en futuras entregas. De aquí "jalamos" las tareas a la columna "To do" cuando decidimos trabajarlas.
- **To do:** Esta columna contiene las tareas que están planificadas para ser realizadas en esta entrega.
- **Doing:** Aquí están las tareas en las que estamos trabajando activamente en este momento.
- **Done:** Esta columna es para las tareas que ya están terminadas. Una vez que una tarea se mueve a esta lista, sabemos que ya no hay que preocuparse por ella.

Link Trello: Tablero Kanban Equipo 4

6. Requisitos

6.1. Requisitos funcionales de alto nivel

Código	Requisitos funcionales
RQF001	Nombre: Extracción, Verificación y Consolidación de Datos (EVC)
	Descripción: Extracción de datos de los diferentes vendors de información, verificación de información y consolidación en base de datos para validación
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales
RQF002	Nombre: Gestión y Validación de Datos (GDV)
	Descripción: Uso de base de datos resultado de EVC donde se utilicen las reglas de validación para brindar el estado de validación de precios de cierre
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales
RQF003	Nombre: Visualización de Datos (DV)
	Descripción: Integración de resultados a una aplicación de visualización para facilitar la interpretación de datos y su identificación sencilla, entrega de reportes y trazabilidad
	Usuarios: Usuario de Price Data

6.2. Requisitos funcionales de bajo nivel

Código	Requisitos funcionales de bajo nivel
RQF_LOW01	Nombre: Registrar division
	Descripción: El sistema debe permitir registrar una division
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW02	Nombre: Consultar division
	Descripción: El sistema debe permitir consultar una division
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW03	Nombre: Modificar division
	Descripción: El sistema debe permitir modificar una division
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW04	Nombre: Eliminar division
	Descripción: El sistema debe permitir eliminar una division
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW05	Nombre: Registrar usuario
	Descripción: El sistema debe permitir registrar un usuario
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW06	Nombre: Consultar usuario
	Descripción: El sistema debe permitir consultar un usuario
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW07	Nombre: Modificar usuario
	Descripción: El sistema debe permitir modificar un usuario
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW08	Nombre: Eliminar usuario
	Descripción: El sistema debe permitir eliminar un usuario
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW09	Nombre: Registrar resultado
	Descripción: El sistema debe permitir registrar un resultado
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW10	Nombre: Consultar resultado
	Descripción: El sistema debe permitir consultar un resultado
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW11	Nombre: Modificar resultado
	Descripción: El sistema debe permitir modificar un resultado
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW12	Nombre: Eliminar resultado
	Descripción: El sistema debe permitir eliminar un resultado
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW13	Nombre: Registrar activo
	Descripción: El sistema debe permitir registrar un activo
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW14	Nombre: Consultar activo
	Descripción: El sistema debe permitir consultar un activo
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW15	Nombre: Modificar activo
	Descripción: El sistema debe permitir modificar un activo
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW16	Nombre: Eliminar activo
	Descripción: El sistema debe permitir eliminar un activo
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW17	Nombre: Registrar vendor
	Descripción: El sistema debe permitir registrar un vendor
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW18	Nombre: Consultar vendor
	Descripción: El sistema debe permitir consultar un vendor
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW19	Nombre: Modificar vendor
	Descripción: El sistema debe permitir modificar un vendor
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW20	Nombre: Eliminar vendor
	Descripción: El sistema debe permitir eliminar un vendor
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW21	Nombre: Consultar activo por vendor
	Descripción: El sistema debe permitir consultar un activo filtrandolo segun el vendor
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW22	Nombre: Consultar jerarquia entre los vendor
	Descripción: El sistema debe permitir consultar la jerarquía de importancia entre vendors de información
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW23	Nombre: Asignar categoria correspondiente a los precios
	Descripción: El sistema debe realizar la comparación de datos de precios de cierre entre los 3 vendors y asignarle su categoría correspondiente
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW24	Nombre: Registrar validacion manual
	Descripción: El sistema debe permitir el mecanismo de validación manual en caso de ser necesario
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW25	Nombre: Contar validación por tipo
	Descripción: El sistema debe realizar el conteo de cada tipo de validación de datos
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW26	Nombre: Registrar resultados de validación
	Descripción: El sistema debe guardar automáticamente el resultado de cada proceso de validación, incluyendo el estado asignado y la fecha en que se realizó
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW27	Nombre: Consultar errores por fecha
	Descripción: El sistema debe permitir consultar los errores guardados filtrando por fecha
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW28	Nombre: Consultar errores por proveedor
	Descripción: El sistema debe permitir consultar los errores guardados filtrando por proveedor
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW29	Nombre: Filtrar datos
	Descripción: permitir filtrar por vendor, tipo de activo y rango de timestamp
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW30	Nombre: Contar validación por tipo
	Descripción: El sistema debe mostrar al usuario las inconsistencias en los precios invalidados y su diferencia frente a las reglas de validación
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW31	Nombre: Identificar inconsistencias de validación
	Descripción: El sistema debe mostrar al usuario las inconsistencias en los precios invalidados y su diferencia frente a las reglas de validación
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW32	Nombre: Revertir validaciones manuales
	Descripción: El sistema debe permitir revertir una validación manual y restaurar su estado anterior, registrando la acción con el usuario que la ejecutó, la fecha y el motivo de reversión
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW33	Nombre: Consultar datos para visualización
	Descripción: El sistema debe poder consultar la base de datos final
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW34	Nombre: Vizualizar validaciones por tipo
	Descripción: El sistema debe mostrar el consolidado validaciones por tipo
	Usuarios: Usuario de Equity Core

Código	Requisitos funcionales de bajo nivel
RQF_LOW35	Nombre: Filtrar activos por vendor
	Descripción: El sistema debe permitir consultar activos filtrandolos por vendor
	Usuarios: Usuario de Equity Core

Código	Requisitos funcionales de bajo nivel
RQF_LOW36	Nombre: Filtrar activos por tipo
	Descripción: El sistema debe permitir consultar activos filtrandolos por tipo
	Usuarios: Usuario de Equity Core

Código	Requisitos funcionales de bajo nivel
RQF_LOW37	Nombre: Filtrar activos por timestamp
	Descripción: El sistema debe permitir consultar activos filtrandolos por timestamp
	Usuarios: Usuario de Equity Core

Código	Requisitos funcionales de bajo nivel
RQF_LOW38	Nombre: Calcular estadísticas y frecuencia por vendor
	Descripción: El sistema debe mostrar las estadísticas de inconsistencias de cada vendor y su frecuencia
	Usuarios: Usuario de Equity Core

Código	Requisitos funcionales de bajo nivel
RQF_LOW39	Nombre: Modificación controlada de validaciones
	Descripción: El sistema GDV debe permitir modificar los resultados de validaciones manuales de forma controlada, asegurando que los cambios queden registrados con trazabilidad y sin afectar otras validaciones
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW40	Nombre: Consulta historico de validaciones
	Descripción: El sistema debe permitir consultar versiones anteriores de los datos validados, conservadas en la base de datos, para comparar cambios realizados en diferentes fechas
	Usuarios: Usuario de Price Data

Código	Requisitos funcionales de bajo nivel
RQF_LOW41	Nombre: Eliminar validaciones
	Descripción: El sistema debe permitir eliminar registros de validación
	Usuarios: Usuario de Price Data

6.3. Requisitos no funcionales

Código	Requisitos no funcionales
NoRQF01	Nombre: Acceso seguro a configuración
	Descripción: El sistema debe garantizar que solo usuarios autenticados y autorizados puedan acceder a la configuración de reglas de validación.

Código	Requisitos no funcionales
NoRQF02	Nombre: Comunicación cifrada con vendors
	Descripción: Toda la comunicación entre el sistema y las fuentes de datos externas (vendors) debe estar cifrada mediante protocolos seguros.

Código	Requisitos no funcionales
NoRQF03	Nombre: Trazabilidad y control de registros
	Descripción: Los registros de validación deben almacenarse con control de acceso y trazabilidad de cambios para auditoría.

Código	Requisitos no funcionales
NoRQF04	Nombre: Documentación clara del código
	Descripción: El código y las reglas de validación deben estar documentados con comentarios claros y actualizados.

Código	Requisitos no funcionales
NoRQF05	Nombre: Agilidad en incorporación de reglas
	Descripción: La incorporación de una nueva regla de validación no debe requerir más de 4 horas de trabajo técnico.

Código	Requisitos no funcionales
NoRQF06	Nombre: Actualizaciones con mínima interrupción
	Descripción: Las actualizaciones del sistema deben poder realizarse sin interrumpir más de 15 minutos de servicio.

Código	Requisitos no funcionales
NoRQF07	Nombre: Disponibilidad mínima
	Descripción: El sistema debe estar disponible para los usuarios al menos en un 99.5 % del tiempo mensual.

Código	Requisitos no funcionales
NoRQF08	Nombre: Continuidad ante fallo de un vendor
	Descripción: En caso de fallo de un vendor, el sistema debe continuar operando con los proveedores restantes sin interrumpir el flujo de validación.

Código	Requisitos no funcionales
NoRQF09	Nombre: Recuperación máxima
	Descripción: El tiempo máximo de recuperación tras una caída no debe superar las 2 horas

Código	Requisitos no funcionales
NoRQF10	Nombre: Interfaz accesibles configuración
	Descripción: La interfaz de usuario debe permitir configurar nuevas reglas de validación sin necesidad de conocimientos de programación avanzados.

Código	Requisitos no funcionales
NoRQF11	Nombre: Mensajes de error claros y comprensibles
	Descripción: Los mensajes de error o alerta deben ser claros y comprensibles, indicando el motivo de la invalidación y la posible causa (ej. diferencias en unidades, decimales, formato).

Código	Requisitos no funcionales
NoRQF12	Nombre: Procesamiento masivo
	Descripción: El sistema debe procesar y validar al menos 10,000 precios de activos en menos de 1 minuto en condiciones normales de operación.

Código	Requisitos no funcionales
NoRQF13	Nombre: Tiempo de respuesta por activo
	Descripción: El tiempo máximo de respuesta para la validación de un lote de precios no debe superar los 2 segundos por activo.

7. Historias de Usuario para RQF seleccionados

Historia de Usuario	
Número: 01	Usuario: Analista de Price Data de Solactive
Nombre Historia: Registrar division	
Prioridad: Alta	
Descripción: "Siendo Analista de Price Data, quiero poder registrar nuevas divisiones en el sistema para organizar y categorizar de manera efectiva"	
Criterios de Aceptacion: Escenario 1: Registrar división ■ Dado que el analista de Price Data está en la página de gestión de divisiones ■ Y hace clic en el botón "Nueva División" ■ Cuando ingresa un nombre "División Mercados Emergentes" ■ Y hace clic en el botón "Guardar" ■ Entonces el sistema debe crear la nueva división en la base de datos ■ Y debe mostrar el mensaje de confirmación "División registrada exitosamente" ■ Y debe redirigir a la lista de divisiones ■ Y la nueva división debe aparecer en la lista con todos los datos ingresados	

Historia de Usuario	
Número: 02	Usuario: Analista de Price Data de Solactive
Nombre Historia: Consultar división	
Prioridad: Alta	
Descripción: "Como Analista de Price Data, quiero poder consultar los detalles de una división existente para verificar su información y utilizarla en mis análisis"	
Criterios de Aceptacion: Escenario 1: Consultar división existente desde la lista ■ Dado que el analista está en la página de listado de divisiones ■ Y existe al menos una división registrada en el sistema ■ Cuando el analista hace clic en el icono de "Ver" / "Consultar" de una división específica ■ Entonces el sistema debe mostrar la página de detalles de la división ■ Y debe mostrar todos los campos de información: código, nombre, descripción y fecha de creación ■ Y la información debe coincidir exactamente con los datos registrados	

Historia de Usuario	
Número: 03	Usuario: Analista de Price Data de Solactive
Nombre Historia: Modificar división	
Prioridad: Alta	
Descripción: "Como Analista de Price Data, quiero poder modificar la información de una división existente para actualizar sus datos cuando sea necesario"	
Criterios de Aceptacion: Escenario 1: Modificar datos básicos de una división existente ■ Dado que el analista está en la página de detalles de la división "1" ■ Y hace clic en el botón "Editar" ■ Cuando modifica el nombre de "División Mercados Emergente" a "División Mercados Globales" ■ Y hace clic en el botón "Guardar Cambios" ■ Entonces el sistema debe actualizar la división en la base de datos ■ Y debe mostrar el mensaje "División actualizada exitosamente" ■ Y debe redirigir a la página de detalles de la división ■ Y debe mostrar los nuevos datos actualizados	

Historia de Usuario	
Número: 04	Usuario: Analista de Price Data de Solactive
Nombre Historia: Eliminar división	
Prioridad: Alta	
Descripción: "Como Analista de Price Data, quiero poder eliminar divisiones que ya no sean necesarias para mantener la base de datos organizada y actualizada"	
Criterios de Aceptacion: Escenario 1: Eliminar división sin dependencias ■ Dado que el analista está en la página de detalles de la división "5" ■ Y la división no tiene relaciones con otros datos del sistema ■ Cuando hace clic en el botón "Eliminar División" ■ Y confirma la acción en el diálogo "¿Está seguro de eliminar la división 5?" ■ Entonces el sistema debe eliminar la división de la base de datos ■ Y debe mostrar el mensaje "División eliminada exitosamente" ■ Y debe redirigir a la lista de divisiones ■ Y la división eliminada no debe aparecer en el listado	

Historia de Usuario	
Número: 05	Usuario: Administrador del sistema
Nombre Historia: Registrar usuario	
Prioridad: Alta	
Descripción: 'Como Administrador del Sistema, quiero poder registrar nuevos usuarios para grantizarles acceso al sistema de Price Data'	
Criterios de Aceptacion: Escenario 1: Registrar usuario ■ Dado que el administrador está en el listado de usuarios ■ Y existe el usuario "Juan Pérez.^{en} el sistema ■ Cuando hace clic en "Ver Detalles" del usuario ■ Entonces el sistema debe mostrar:	

Historia de Usuario	
Número: 06	Usuario: Administrador del sistema
Nombre Historia: Consultar usuario	
Prioridad: Alta	
Descripción: "Como Administrador del Sistema, quiero poder consultar la información detallada de cualquier usuario para verificar sus datos"	
Criterios de Aceptacion: Escenario 1: Consultar usuario existente ■ Dado que el administrador está en el listado de usuarios ■ Y existe el usuario "Juan Pérez.^{en} el sistema ■ Cuando hace clic en "Ver Detalles" del usuario ■ Entonces el sistema debe mostrar: ID, Email, Nombre, Contraseña y División	

Historia de Usuario	
Número: 07	Usuario: Administrador del sistema
Nombre Historia: Modificar usuario	
Prioridad: Alta	
Descripción: "Como Administrador del Sistema, quiero poder modificar la información de usuarios existentes para actualizar sus datos cuando sea necesario"	
Criterios de Aceptacion: Escenario 1: Modificar datos básicos de una división existente ■ Dado que el administrador está editando el usuario "Juan Pérez" ■ Cuando cambia el nombre a "Juan Carlos Pérez" ■ Y cambia la división a "2 - Mercados Europeos" ■ Y hace clic en "Guardar Cambios" ■ Entonces el sistema debe actualizar el usuario ■ Y debe mostrar "Usuario actualizado exitosamente" ■ Y debe registrar el cambio en el historial	

Historia de Usuario	
Número: 08	Usuario: Analista de Price Data de Solactive
Nombre Historia: Eliminar usuario	
Prioridad: Alta	
Descripción: "Como Administrador del Sistema, quiero poder eliminar usuarios que ya no necesiten acceso al sistema para mantener la seguridad y organización"	
Criterios de Aceptacion: Escenario 1: Eliminar usuario ▪ Dado que el usuario "Temporal User" no tiene registros creados ▪ Y no tiene datos asociados en el sistema ▪ Cuando el administrador hace clic en "Eliminar Usuario" ▪ Y confirma la acción en el diálogo ▪ Entonces el sistema debe eliminar el usuario permanentemente ▪ Y debe mostrar "Usuario eliminado exitosamente" ▪ Y debe removerlo de todos los listados	

Historia de Usuario	
Número: 09	Usuario: Usuario de Price Data
Nombre Historia: Registrar resultado	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder registrar un resultado para almacenar el estado de validación de los precios de los activos"	
Criterios de Aceptacion: Escenario 1: Registrar resultado de validación automática ▪ Dado que el sistema ha comparado los precios de un activo entre los tres vendors ▪ Y se ha determinado el estado de validación ▪ Cuando el sistema guarda automáticamente el resultado ▪ Entonces debe registrar el idActivoFK, observacionResultado y fecha de validación ▪ Y debe almacenar el estado asignado (VALIDATED/SEMI-VALIDATED/UNVALIDATED) ▪ Y debe mostrar "Resultado registrado exitosamente"	

Historia de Usuario	
Número: 10	Usuario: Usuario de Price Data
Nombre Historia: Consultar resultado	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder consultar un resultado para verificar el estado de validación de los precios de los activos"	
Criterios de Aceptacion: Escenario 1: Consultar resultado existente ■ Dado que existe un resultado registrado en el sistema ■ Cuando el usuario accede al listado de resultados ■ Y selecciona un resultado específico ■ Entonces el sistema debe mostrar: idResultado, idActivoFK, observacionResultado y estado de validación ■ Y debe mostrar la fecha y hora de registro ■ Y la información debe coincidir con los datos almacenados	

Historia de Usuario	
Número: 11	Usuario: Usuario de Price Data
Nombre Historia: Modificar resultado	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder modificar un resultado para actualizar el estado de validación cuando sea necesario"	
Criterios de Aceptacion: Escenario 1: Modificar observación de resultado ■ Dado que el usuario está en la página de detalles de un resultado ■ Y hace clic en el botón "Editar" ■ Cuando modifica la observación "Diferencia en decimales" a "Diferencia corregida manualmente" ■ Y hace clic en el botón "Guardar Cambios" ■ Entonces el sistema debe actualizar el resultado en la base de datos ■ Y debe mostrar "Resultado actualizado exitosamente" ■ Y debe registrar el cambio en el historial	

Historia de Usuario	
Número: 12	Usuario: Usuario de Price Data
Nombre Historia: Eliminar resultado	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder eliminar un resultado para mantener la base de datos actualizada"	
Criterios de Aceptacion: Escenario 1: Eliminar resultado sin dependencias ■ Dado que el resultado no tiene relaciones con otros datos del sistema ■ Cuando el usuario hace clic en el botón "Eliminar Resultado" ■ Y confirma la acción en el diálogo ■ Entonces el sistema debe eliminar el resultado de la base de datos ■ Y debe mostrar "Resultado eliminado exitosamente" ■ Y debe removerlo de todos los listados	

Historia de Usuario	
Número: 13	Usuario: Usuario de Price Data
Nombre Historia: Registrar activo	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder registrar un activo para incluir nuevos instrumentos financieros en el sistema de validación"	
Criterios de Aceptacion: Escenario 1: Registrar activo con datos completos ■ Dado que el usuario está en la página de gestión de activos ■ Y hace clic en el botón "Nuevo Activo" ■ Cuando ingresa precioActivo 150.75, divisaActivo "USD", tickerUniversal "AAPL", regionActivo "NORTH AMERICA", claseActivo "Acción" ■ Y hace clic en "Guardar" ■ Entonces el sistema debe crear el activo en la base de datos ■ Y debe mostrar "Activo registrado exitosamente" ■ Y debe redirigir al listado de activos	

Historia de Usuario	
Número: 14	Usuario: Usuario de Price Data
Nombre Historia: Consultar activo	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder consultar un activo para verificar su información y precios registrados"	
Criterios de Aceptacion: Escenario 1: Consultar activo por ticker universal ■ Dado que existe el activo con tickerUniversal "AAPL" ■ Cuando el usuario ingresa "AAPL" en el campo de búsqueda ■ Y hace clic en "Buscar" ■ Entonces el sistema debe mostrar: precioActivo, divisaActivo, regionActivo, claseActivo ■ Y debe mostrar el timestampRecepcion ■ Y la información debe coincidir con los datos registrados	

Historia de Usuario	
Número: 15	Usuario: Usuario de Price Data
Nombre Historia: Modificar activo	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder modificar un activo para actualizar su información cuando sea necesario"	
Criterios de Aceptacion: Escenario 1: Actualizar precio de activo ■ Dado que el usuario está editando el activo "AAPL" ■ Cuando modifica el precioActivo de 150.75 a 152.30 ■ Y actualiza el timestampRecepcion ■ Y hace clic en "Guardar Cambios" ■ Entonces el sistema debe actualizar el activo en la base de datos ■ Y debe mostrar "Activo actualizado exitosamente" ■ Y debe registrar el cambio en el historial	

Historia de Usuario	
Número: 16	Usuario: Usuario de Price Data
Nombre Historia: Eliminar activo	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder eliminar un activo para mantener la base de datos actualizada"	
Criterios de Aceptacion: Escenario 1: Eliminar activo sin resultados asociados ■ Dado que el activo no tiene resultados de validación asociados ■ Cuando el usuario hace clic en "Eliminar Activo" ■ Y confirma la acción en el diálogo ■ Entonces el sistema debe eliminar el activo de la base de datos ■ Y debe mostrar "Activo eliminado exitosamente" ■ Y debe removerlo de todos los listados	

Historia de Usuario	
Número: 17	Usuario: Usuario de Price Data
Nombre Historia: Registrar vendor	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder registrar un vendor para incluir nuevas fuentes de datos en el sistema"	
Criterios de Aceptacion: Escenario 1: Registrar vendor con datos completos ■ Dado que el usuario está en la página de gestión de vendors ■ Y hace clic en el botón "Nuevo Vendor" ■ Cuando ingresa nombre "ICE", emailSoporte "soporte@ice.com", importancia 1 ■ Y hace clic en "Guardar" ■ Entonces el sistema debe crear el vendor en la base de datos ■ Y debe mostrar "Vendor registrado exitosamente" ■ Y debe redirigir al listado de vendors	

Historia de Usuario	
Número: 18	Usuario: Usuario de Price Data
Nombre Historia: Consultar vendor	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder consultar un vendor para verificar su información y nivel de importancia"	
Criterios de Aceptacion: Escenario 1: Consultar vendor existente ■ Dado que existe el vendor "ICE" en el sistema ■ Cuando el usuario selecciona el vendor "ICE" del listado ■ Entonces el sistema debe mostrar: nombre, emailSoporte, importancia ■ Y debe mostrar el idVendor ■ Y la información debe coincidir con los datos registrados	

Historia de Usuario	
Número: 19	Usuario: Usuario de Price Data
Nombre Historia: Modificar vendor	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder modificar un vendor para actualizar su información cuando sea necesario"	
Criterios de Aceptacion: Escenario 1: Actualizar información de vendor ■ Dado que el usuario está editando el vendor "ICE" ■ Cuando modifica el emailSoporte a "support@ice.com" ■ Y actualiza la importancia a 2 ■ Y hace clic en "Guardar Cambios" ■ Entonces el sistema debe actualizar el vendor en la base de datos ■ Y debe mostrar "Vendor actualizado exitosamente" ■ Y debe registrar el cambio en el historial	

Historia de Usuario	
Número: 20	Usuario: Usuario de Price Data
Nombre Historia: Eliminar vendedor	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder eliminar un vendedor para mantener la base de datos actualizada"	
Criterios de Aceptacion: Escenario 1: Eliminar vendedor sin activos asociados ■ Dado que el vendedor no tiene activos asociados en DetalleRecepcion ■ Cuando el usuario hace clic en "Eliminar Vendedor" ■ Y confirma la acción en el diálogo ■ Entonces el sistema debe eliminar el vendedor de la base de datos ■ Y debe mostrar "Vendedor eliminado exitosamente" ■ Y debe removerlo de todos los listados	

Historia de Usuario	
Número: 21	Usuario: Usuario de Price Data
Nombre Historia: Consultar activo por vendedor	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder consultar un activo filtrándolo según el vendedor para analizar la información por proveedor"	
Criterios de Aceptacion: Escenario 1: Consultar activos por vendedor específico ■ Dado que existen activos asociados al vendedor "ICE" ■ Cuando el usuario selecciona "ICE" en el filtro de vendedores ■ Y hace clic en "Filtrar" ■ Entonces el sistema debe mostrar solo los activos provenientes de ICE ■ Y debe mostrar el precioActivo y timestampRecepcion de cada activo ■ Y no debe mostrar activos de otros vendedores	

Historia de Usuario	
Número: 22	Usuario: Usuario de Price Data
Nombre Historia: Consultar jerarquía entre los vendors	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder consultar la jerarquía de importancia entre vendors de información para entender el orden de prioridad"	
Criterios de Aceptacion: Escenario 1: Consultar jerarquía de vendors ■ Dado que existen los vendors ICE, FactSet y EDI en el sistema ■ Cuando el usuario accede a la sección "Jerarquía de Vendors" ■ Entonces el sistema debe mostrar la lista ordenada por importancia: ICE (1), FactSet (2), EDI (3) ■ Y debe mostrar el nombre y nivel de importancia de cada vendor ■ Y el orden debe coincidir con la configuración del sistema	

Historia de Usuario	
Número: 23	Usuario: Usuario de Price Data
Nombre Historia: Asignar categoría correspondiente a los precios	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero que el sistema realice la comparación de datos de precios de cierre entre los 3 vendors y asigne la categoría correspondiente"	
Criterios de Aceptacion: Escenario 1: Asignar categoría VALIDATED ■ Dado que los tres vendors reportan el mismo precio para un activo ■ Cuando el sistema ejecuta el proceso de comparación ■ Entonces debe asignar la categoría "VALIDATED" al resultado ■ Y debe registrar el resultado con el estado correspondiente ■ Y debe mostrar el conteo de precios validated en el reporte	

Historia de Usuario	
Número: 24	Usuario: Usuario de Price Data
Nombre Historia: Registrar validación manual	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder realizar validación manual en caso de ser necesario para corregir resultados automáticos"	
Criterios de Aceptacion: Escenario 1: Registrar validación manual exitosa ■ Dado que un resultado automático está marcado como UNVALIDATED ■ Cuando el usuario selecciona "Validación Manual" ■ Y cambia el estado a "USER-VALIDATION" ■ Y ingresa la observación "Precio validado manualmente por discrepancias menores" ■ Y hace clic en "Guardar" ■ Entonces el sistema debe actualizar el estado a USER-VALIDATION ■ Y debe registrar la observación y el usuario que realizó la validación ■ Y debe mostrar "Validación manual registrada exitosamente"	

Historia de Usuario	
Número: 25	Usuario: Usuario de Price Data
Nombre Historia: Contar validación por tipo	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero que el sistema realice el conteo de cada tipo de validación de datos para generar reportes estadísticos"	
Criterios de Aceptacion: Escenario 1: Generar conteo por tipo de validación ■ Dado que existen resultados con diferentes estados de validación ■ Cuando el usuario solicita el reporte de conteo por tipo ■ Entonces el sistema debe mostrar: cantidad de VALIDATED, SEMI-VALIDATED, UNVALIDATED, USER-VALIDATION ■ Y debe calcular los porcentajes de cada tipo ■ Y debe mostrar la fecha del reporte generado	

Historia de Usuario	
Número: 26	Usuario: Usuario de Price Data
Nombre Historia: Registrar resultados de validación	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero que el sistema guarde automáticamente el resultado de cada proceso de validación, incluyendo el estado asignado y la fecha en que se realizó"	
Criterios de Aceptacion: Escenario 1: Registro automático de resultado de validación ■ Dado que el sistema ha completado un proceso de validación ■ Cuando se determina el estado final del activo ■ Entonces el sistema debe guardar automáticamente: idActivoFK, estado asignado, fecha y hora de validación ■ Y debe registrar cualquier observación relevante ■ Y debe confirmar el almacenamiento en la base de datos	

Historia de Usuario	
Número: 27	Usuario: Usuario de Price Data
Nombre Historia: Consultar errores por fecha	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder consultar los errores guardados filtrando por fecha para analizar tendencias temporales"	
Criterios de Aceptacion: Escenario 1: Filtrar errores por rango de fechas ■ Dado que existen errores registrados en diferentes fechas ■ Cuando el usuario selecciona fecha inicio "2024-01-01" y fecha fin "2024-01-31" ■ Y hace clic en "Filtrar" ■ Entonces el sistema debe mostrar solo los errores del mes de enero 2024 ■ Y debe mostrar el tipo de error y fecha de ocurrencia ■ Y debe incluir el conteo total de errores en el período	

Historia de Usuario	
Número: 28	Usuario: Usuario de Price Data
Nombre Historia: Consultar errores por proveedor	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder consultar los errores guardados filtrando por proveedor para identificar problemas específicos"	
Criterios de Aceptacion: Escenario 1: Filtrar errores por vendor específico ■ Dado que existen errores asociados a diferentes vendors ■ Cuando el usuario selecciona "FactSet" en el filtro de proveedores ■ Y hace clic en "Filtrar" ■ Entonces el sistema debe mostrar solo los errores relacionados con FactSet ■ Y debe mostrar el tipo de error y activo afectado ■ Y debe incluir el conteo total de errores del proveedor	

Historia de Usuario	
Número: 29	Usuario: Usuario de Price Data
Nombre Historia: Filtrar datos	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder filtrar por vendor, tipo de activo y rango de timestamp para realizar análisis específicos"	
Criterios de Aceptacion: Escenario 1: Aplicar múltiples filtros simultáneamente ■ Dado que el usuario necesita filtrar datos específicos ■ Cuando selecciona vendor "ICE", tipo de activo "Acción", y rango de timestamp "2024-01-01" a "2024-01-31" ■ Y hace clic en "Aplicar Filtros" ■ Entonces el sistema debe mostrar solo los activos de tipo Acción de ICE en enero 2024 ■ Y debe aplicar todos los filtros simultáneamente ■ Y debe mostrar el número de resultados filtrados	

Historia de Usuario	
Número: 30	Usuario: Usuario de Price Data
Nombre Historia: Identificar inconsistencias de validación	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero que el sistema muestre al usuario las inconsistencias en los precios invalidados y su diferencia frente a las reglas de validación"	
Criterios de Aceptacion: Escenario 1: Identificación de inconsistencias en precios ■ Dado que existen precios marcados como UNVALIDATED ■ Cuando el usuario consulta el reporte de inconsistencias ■ Entonces el sistema debe mostrar: activo, diferencia entre precios de vendedores, regla de validación incumplida ■ Y debe indicar el vendedor que causó la invalidación ■ Y debe mostrar la magnitud y tipo de discrepancia	

Historia de Usuario	
Número: 31	Usuario: Usuario de Price Data
Nombre Historia: Revertir validaciones manuales	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder revertir una validación manual y restaurar su estado anterior, registrando la acción con el usuario que la ejecutó, la fecha y el motivo de reversión"	
Criterios de Aceptacion: Escenario 1: Revertir validación manual exitosamente ■ Dado que existe una validación manual con estado USER-VALIDATION ■ Cuando el usuario selecciona "Revertir Validación" ■ Y ingresa el motivo "Error en validación manual" ■ Y confirma la acción ■ Entonces el sistema debe restaurar el estado anterior (UNVALIDATED/SEMI-VALIDATED) ■ Y debe registrar usuario, fecha y motivo de reversión ■ Y debe mostrar "Validación revertida exitosamente"	

Historia de Usuario	
Número: 32	Usuario: Usuario de Price Data
Nombre Historia: Consultar datos para visualización	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder consultar la base de datos final para obtener datos estructurados para visualización"	
Criterios de Aceptacion: Escenario 1: Consulta exitosa de datos para visualización ■ Dado que la base de datos contiene resultados de validación ■ Cuando el usuario solicita datos para visualización ■ Entonces el sistema debe retornar: activos, estados de validación, fechas, vendors ■ Y debe estructurar los datos en formato compatible con herramientas de visualización ■ Y debe incluir metadatos necesarios para gráficos y reportes	

Historia de Usuario	
Número: 33	Usuario: Usuario de Equity Core
Nombre Historia: Visualizar validaciones por tipo	
Prioridad: Alta	
Descripción: "Como Usuario de Equity Core, quiero que el sistema muestre el consolidado de validaciones por tipo para análisis de calidad de datos"	
Criterios de Aceptacion: Escenario 1: Visualización de consolidado por tipo ■ Dado que existen validaciones de diferentes tipos ■ Cuando el usuario accede al dashboard de validaciones ■ Entonces el sistema debe mostrar gráfico con distribución: VALIDATED, SEMI-VALIDATED, UNVALIDATED, USER-VALIDATION ■ Y debe mostrar números absolutos y porcentajes ■ Y debe permitir filtrar por fecha y vendor	

Historia de Usuario	
Número: 34	Usuario: Usuario de Equity Core
Nombre Historia: Filtrar activos por vendor	
Prioridad: Alta	
Descripción: "Como Usuario de Equity Core, quiero poder consultar activos filtrándolos por vendor para análisis específicos por proveedor"	
Criterios de Aceptacion: Escenario 1: Filtrado de activos por vendor ■ Dado que existen activos de múltiples vendors ■ Cuando el usuario selecciona "EDI" en el filtro de vendors ■ Y aplica el filtro ■ Entonces el sistema debe mostrar solo activos provenientes de EDI ■ Y debe mostrar información completa de cada activo ■ Y debe indicar el número total de activos del vendor	

Historia de Usuario	
Número: 35	Usuario: Usuario de Equity Core
Nombre Historia: Filtrar activos por tipo	
Prioridad: Alta	
Descripción: "Como Usuario de Equity Core, quiero poder consultar activos filtrándolos por tipo para análisis por clase de instrumento"	
Criterios de Aceptacion: Escenario 1: Filtrado de activos por tipo ■ Dado que existen activos de diferentes tipos ■ Cuando el usuario selecciona "Bono" en el filtro de tipo de activo ■ Y aplica el filtro ■ Entonces el sistema debe mostrar solo activos de tipo Bono ■ Y debe mostrar precios y características específicas de bonos ■ Y debe agrupar por emisor y fecha de vencimiento	

Historia de Usuario	
Número: 36	Usuario: Usuario de Equity Core
Nombre Historia: Filtrar activos por timestamp	
Prioridad: Alta	
Descripción: "Como Usuario de Equity Core, quiero poder consultar activos filtrándolos por timestamp para análisis temporales"	
Criterios de Aceptacion: Escenario 1: Filtrado de activos por rango temporal ■ Dado que existen activos con diferentes timestamps ■ Cuando el usuario selecciona rango "2024-01-01" a "2024-01-15" ■ Y aplica el filtro ■ Entonces el sistema debe mostrar solo activos del período seleccionado ■ Y debe ordenar por fecha de recepción ■ Y debe mostrar evolución de precios en el tiempo	

Historia de Usuario	
Número: 37	Usuario: Usuario de Equity Core
Nombre Historia: Calcular estadísticas y frecuencia por vendor	
Prioridad: Alta	
Descripción: "Como Usuario de Equity Core, quiero que el sistema muestre las estadísticas de inconsistencias de cada vendor y su frecuencia para evaluación de calidad"	
Criterios de Aceptacion: Escenario 1: Generación de estadísticas por vendor ■ Dado que existen inconsistencias registradas por vendor ■ Cuando el usuario solicita el reporte de estadísticas ■ Entonces el sistema debe mostrar: vendor, número de inconsistencias, frecuencia, tipo de errores más comunes ■ Y debe calcular porcentaje de error por vendor ■ Y debe comparar performance entre vendors	

Historia de Usuario	
Número: 38	Usuario: Usuario de Price Data
Nombre Historia: Modificación controlada de validaciones	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero que el sistema permita modificar los resultados de validaciones manuales de forma controlada, asegurando que los cambios queden registrados con trazabilidad y sin afectar otras validaciones"	
Criterios de Aceptacion: Escenario 1: Modificación controlada exitosa ▪ Dado que existe una validación manual USER-VALIDATION ▪ Cuando el usuario modifica el estado a SEMI-VALIDATED ▪ Y ingresa justificación "Reevaluación de criterios" ▪ Y confirma la modificación ▪ Entonces el sistema debe actualizar el estado manteniendo trazabilidad ▪ Y debe registrar usuario, fecha y justificación del cambio ▪ Y debe mantener integridad de otras validaciones	

Historia de Usuario	
Número: 39	Usuario: Usuario de Price Data
Nombre Historia: Consultar histórico de validaciones	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder consultar versiones anteriores de los datos validados, conservadas en la base de datos, para comparar cambios realizados en diferentes fechas"	
Criterios de Aceptacion: Escenario 1: Consulta de histórico de cambios ▪ Dado que existen múltiples versiones de una validación ▪ Cuando el usuario selecciona "Ver Histórico" ▪ Entonces el sistema debe mostrar: fechas de modificación, estados anteriores, usuarios que realizaron cambios ▪ Y debe permitir comparar versiones side-by-side ▪ Y debe mostrar diferencias entre versiones	

Historia de Usuario	
Número: 40	Usuario: Usuario de Price Data
Nombre Historia: Eliminar validaciones	
Prioridad: Alta	
Descripción: "Como Usuario de Price Data, quiero poder eliminar registros de validación para mantener la base de datos limpia"	
Criterios de Aceptacion: Escenario 1: Eliminación segura de validación ■ Dado que existe una validación que necesita ser eliminada ■ Cuando el usuario selecciona "Eliminar Validación" ■ Y confirma la acción con justificación ■ Entonces el sistema debe eliminar el registro de validación ■ Y debe registrar la eliminación en el histórico de auditoría ■ Y debe mostrar "Validación eliminada exitosamente"	

8. Flujo de solución

A continuación se presentan los flujos de solución planteados para tratar la problemática de validaciones de precios.

8.1. Opción 1

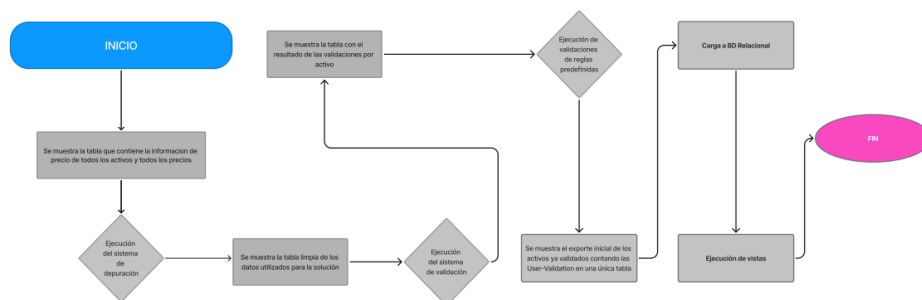


Figura 1: Opción 1 de solución: Construcción de reglas de validación bajo supervisión de la empresa

Este flujo de solución consiste en construir las reglas de validación de la mano de la empresa, para evitar validaciones incorrectas durante el proceso.

8.2. Opción 2

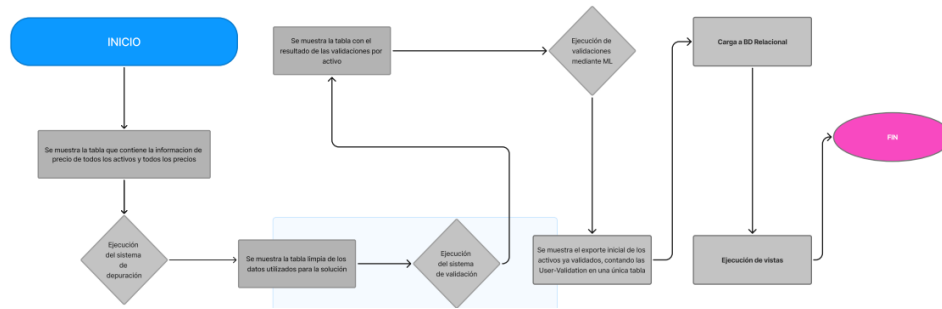


Figura 2: Opción 2 de solución: Uso de algoritmos de clasificación de Machine Learning para detectar las User-Validation y los datos no validados correctos

Este flujo de solución consiste en utilizar el algoritmo de Machine Learning para detectar la clasificación correcta sobre si es un *USER-VALIDATION* o un *UNVALIDATED*

8.3. Solución seleccionada

Se seleccionó la opción 1, porque esta permite evitar problemas de clasificación propios de los algoritmos de Machine Learning, lo cual puede ser costoso para la empresa y afectar su reputación directamente.

9. Diagrama de clases

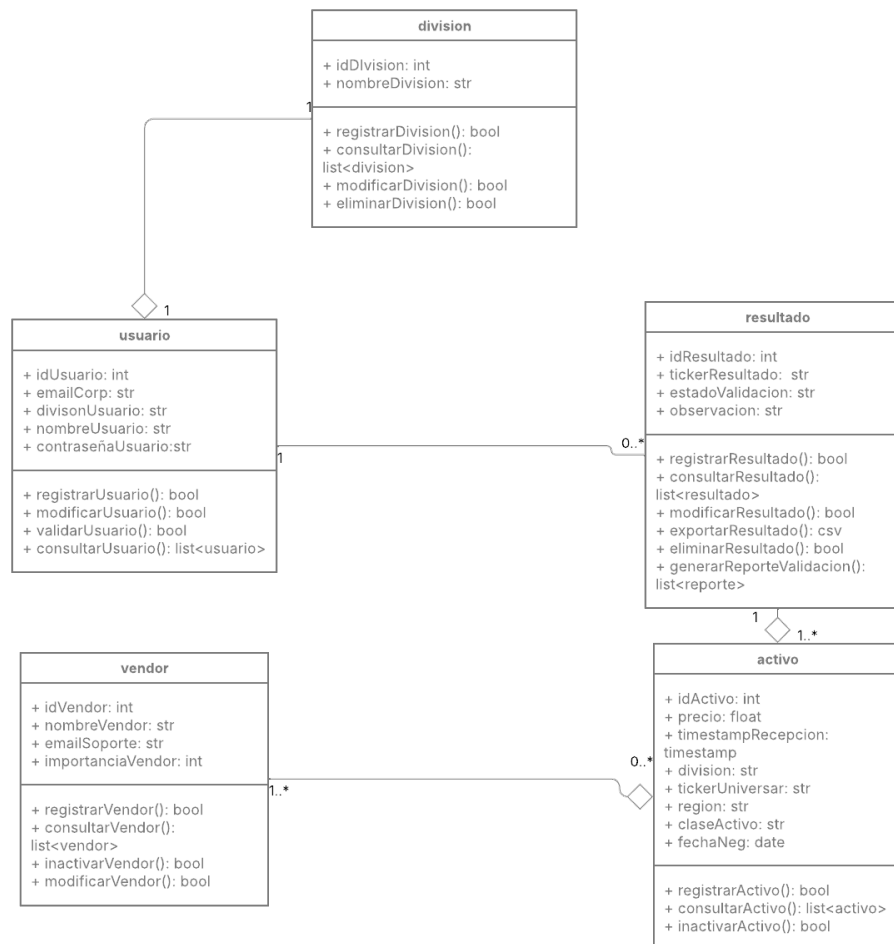


Figura 3: Diagrama de clases para la validación de precios entre activos

10. Modelo Entidad-Relación (MER)

Para poder visualizar al MER, acceder haciendo clic en la siguiente palabra: [MER](#)

11. Modelo físico

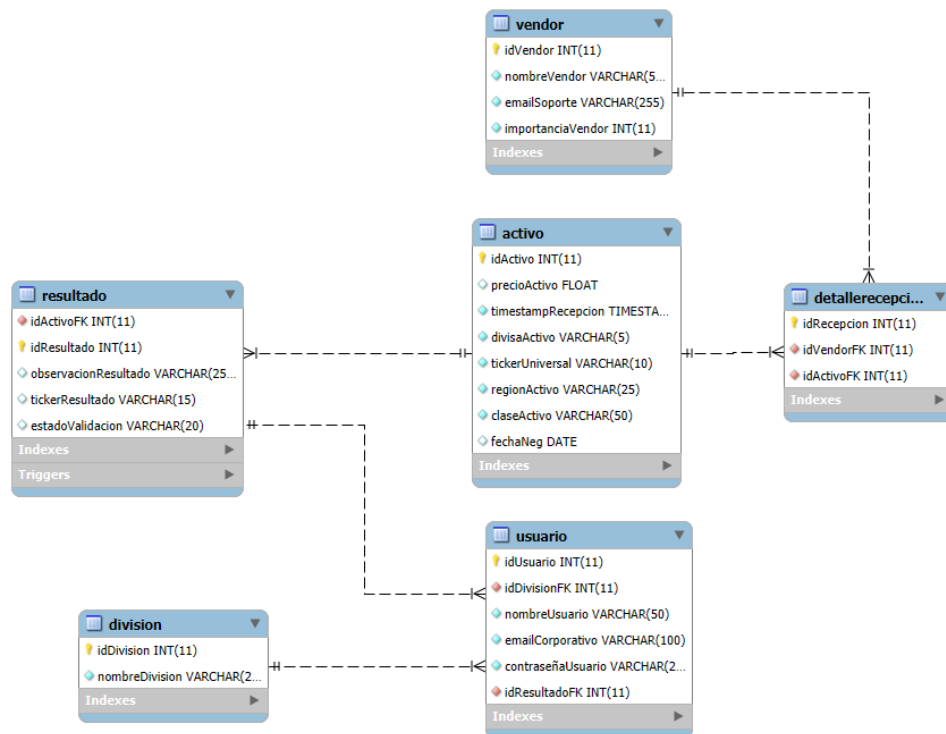


Figura 4: Modelo físico de base de datos relacional

12. Diccionario de datos

Nombre de la tabla	Vendor			
Descripción	Tabla que contiene la información de los diferentes vendors.			
Campo	Tipo de dato	Tamaño	Restricción	Descripción
idVendor	int	—	Primary key – AUTOINCREMENT	Llave primaria del vendor.
nombre Vendor	varchar	50	NOT NULL	Nombre del vendor.
emailSoporte	varchar	255	NOT NULL	Email de contacto al área de soporte del vendor correspondiente.
importancia Vendor	int	—	NOT NULL	Nivel de importancia del vendor (1 es el más alto y 3 el más bajo).

Tabla 1: Vendor

Nombre de la tabla	Activo			
Descripción	Tabla que contiene la información relevante de los activos registrados.			
Campo	Tipo de dato	Tamaño	Restricción	Descripción
idActivo	int	–	Primary key – AUTOINCREMENT	Llave primaria del activo.
precioActivo	float	–	NOT NULL	Precio del activo.
timestamp Recepcion	timestamp	–	NOT NULL	La fecha y hora en la que fue enviada la información del activo brindada por un vendedor específico.
divisaActivo	varchar	5	NOT NULL	Moneda en la que el activo se negocia en el sistema de información de cada vendedor.
ticker Universal	varchar	10	NOT NULL	Identificador universal del activo entre los 3 vendedores de información.
regionActivo	varchar	25	NOT NULL	Región de emisión del activo.
claseActivo	varchar	50	NOT NULL	Tipo de activo negociado (p. ej., acción, bono, moneda).

Tabla 2: Activo

Nombre de la tabla	DetalleRecepcion			
Descripción	Tabla débil que permite relacionar la información de los activos con el vendedor que la brinda.			
Campo	Tipo de dato	Tamaño	Restricción	Descripción
idRecepcion	int	–	Primary key – AUTOINCREMENT	Identificador de la recepción.
idVendorFK	int	–	Foreign Key	Llave foránea del vendedor; indica qué vendedor brindó la información del activo.
idActivoFK	int	–	Foreign Key	Llave foránea del activo; permite relacionar el activo registrado con el vendedor que brindó la información.

Tabla 3: DetalleRecepcion

Nombre de la tabla	Resultado			
Descripción	Tabla que muestra el resultado de las validaciones de la información de precios de los activos brindada por cada vendedor.			
Campo	Tipo de dato	Tamaño	Restricción	Descripción
idResultado	int	–	Primary key – AUTOINCREMENT	Llave primaria del resultado; identifica un resultado para los precios brindados por los 3 vendedores para un activo en la misma fecha de negociación.
idActivoFK	int	–	Foreign Key	Llave foránea del activo; relaciona la información del activo brindada por el vendedor primario.
observacion Resultado	varchar	255	–	Observación asociada a precios invalidados o a validaciones realizadas por usuario, para especificar la razón del resultado.

Tabla 4: Resultado

Nombre de la tabla	Usuario			
Descripción	Tabla que contiene la información relevante de los usuarios que acceden a la base de datos.			
Campo	Tipo de dato	Tamaño	Restricción	Descripción
idUsuario	int	–	Primary key – AUTOINCREMENT	Llave primaria del usuario; identificador único para cada usuario.
idDivisionFK	int	–	Foreign Key – NOT NULL	Llave foránea de la división; identifica la división a la que pertenece el usuario.
nombre Usuario	varchar	50	NOT NULL	Nombre del usuario.
email Coparativo	varchar	100	NOT NULL	Email corporativo correspondiente al usuario.
contraseña Usuario	varchar	250	NOT NULL	Contraseña del usuario.
idResultado FK	int	–	Foreign Key	Llave foránea de los resultados; identifica los resultados ejecutados por el usuario.

Tabla 5: Usuario

Nombre de la tabla	Division			
Descripción	Tabla que muestra la información relevante de las divisiones que tienen acceso a la base de datos.			
Campo	Tipo de dato	Tamaño	Restricción	Descripción
idDivision	int	–	Primary key – AUTOINCREMENT	Llave primaria de la división.
nombre Division	varchar	250	NOT NULL	Nombre de las divisiones que tienen acceso a la base de datos.

Tabla 6: Division

13. Ejecución de Requisitos

13.1. RQF 1 - Insertar Divisiones

Implementación: Inserción de nuevas divisiones en la tabla `division`

```
INSERT INTO division VALUES ("","Cripto"), ("","bons");
```

13.2. RQF 2 - Consultar Divisiones

Implementación: Consulta de todas las divisiones

```
SELECT * FROM division;
```

13.3. RQF 3 - Actualizar División

Implementación: Actualización del nombre de una división

```
UPDATE division SET nombreDivision = 'price-data' WHERE idDivision = 2;
```

13.4. RQF 4 - Eliminar División

Implementación: Eliminación de una división por ID

```
DELETE FROM division WHERE idDivision = 3;
```

13.5. RQF 5 - Insertar Usuario

Implementación: Inserción de nuevo usuario

```
INSERT INTO usuario VALUES ("","2","emilia","emilia@solactiveequity.com",  
                             "emilia1990","1");
```

13.6. RQF 6 - Buscar Usuario por Email

Implementación: Consulta de usuario por correo electrónico

```
SELECT * FROM usuario WHERE correo = 'emilia@solactiveequity.com';
```

13.7. RQF 7 - Actualizar Usuario

Implementación: Actualización de datos de usuario

```
UPDATE usuario SET nombre = 'Carlos Pérez', correo = 'carlos@example.com'  
WHERE idUsuario = 1;
```

13.8. RQF 8 - Eliminar Usuario

Implementación: Eliminación de usuario por email

```
DELETE FROM usuario WHERE correo = 'emilia@solactiveequity.com';
```

13.9. RQF 9 - Insertar Resultado

Implementación: Inserción de nuevo resultado de validación

```
INSERT INTO resultado VALUES (48, "", "", "XHEL", "Validated");
```

13.10. RQF 10 - Consultar Resultados

Implementación: Consulta de todos los resultados

```
SELECT * FROM resultado;
```

13.11. RQF 11 - Consultar Vendors Ordenados

Implementación: Consulta de vendors ordenados por importancia

```
SELECT nombreVendor as "Nombre-Vendor", importanciaVendor as "Jerarquía"  
FROM vendor ORDER BY importanciaVendor asc;
```

13.12. RQF 12 - Eliminar Resultado

Implementación: Eliminación de resultado por ID

```
DELETE FROM resultado WHERE idResultado = 1;
```

13.13. RQF 13 - Insertar Activo

Implementación: Inserción de nuevo activo

```
INSERT INTO activo VALUES ("", "0.235", '2025-11-13 14:22:00', 'EUR',  
                             'XAMS', 'EUROPA', 'share', '2025-09-14');
```

13.14. RQF 14 - Consultar Activo por ID

Implementación: Consulta de activo específico por ID

```
SELECT * FROM activo WHERE idActivo = 10;
```

13.15. RQF 15 - Actualizar Ticker

Implementación: Actualización del ticker de un activo

```
UPDATE activo SET tickerUniversal = 'MSFT' WHERE idActivo = 3;
```

13.16. RQF 16 - Eliminar Activo

Implementación: Eliminación de activo por ID

```
DELETE FROM activo WHERE idActivo = 10;
```

13.17. RQF 17 - Insertar Vendor

Implementación: Inserción de nuevo vendor

```
INSERT INTO vendor VALUES ( ' ', 'Global ', "venodr4@jajaj.com", "4" );
```

13.18. RQF 18 - Consultar Vendor por ID

Implementación: Consulta de vendor específico por ID

```
SELECT * FROM vendor WHERE idVendor = 1;
```

13.19. RQF 19 - Actualizar Vendor

Implementación: Actualización del nombre de un vendor

```
UPDATE vendor SET nombreVendor = 'Bloomberg-LP' WHERE idVendor = 1;
```

13.20. RQF 20 - Eliminar Vendor

Implementación: Eliminación de vendor por ID

```
DELETE FROM vendor WHERE idVendor = 1;
```

14. Requisitos de Procedimientos Almacenados

14.1. RQF 20 - Filtro de Activos

Implementación: Procedimiento para filtrar activos con múltiples criterios

```

DELIMITER $$
CREATE PROCEDURE FiltroActivos(
    IN divisaBusqueda varchar(5),
    IN tickerBusqueda varchar(5),
    IN regionBusqueda varchar(5),
    IN claseActivoBusqueda varchar(25),
    IN fechaBusqueda date)
BEGIN
    SELECT * FROM activo
    WHERE tickerUniversal IN (SELECT tickerUniversal FROM activo
                             WHERE tickerBusqueda IS NULL OR tickerUniversal = tickerBusqueda)
    AND (divisaBusqueda IS NULL OR divisaActivo = divisaBusqueda)
    AND (regionBusqueda IS NULL OR regionActivo = regionBusqueda)
    AND (claseActivoBusqueda IS NULL OR claseActivo = claseActivoBusqueda)
    AND (fechaBusqueda IS NULL OR fechaNeg = fechaBusqueda);
END $$
DELIMITER ;

```

14.2. RQF 21 - Consulta Activos por Vendor

Implementación: Consulta de activos asociados a un vendor específico

```

SELECT a.idActivo, a.tickerUniversal, a.precioActivo, a.timestampRecepcion,
       a.divisaActivo, a.regionActivo, a.claseActivo, v.nombreVendor
FROM activo a
INNER JOIN detalleRecepcion dr ON a.idActivo = dr.idActivoFK
INNER JOIN vendor v ON dr.idVendorFK = v.idVendor
WHERE v.nombreVendor = "ICE";

```

14.3. RQF 22 - Consulta Vendors Ordenados

Implementación: Consulta de vendors ordenados por importancia

```

SELECT idVendor, nombreVendor, emailSoporte, importanciaVendor
FROM vendor
ORDER BY importanciaVendor ASC;

```

14.4. RQF 23 - Comparación de Precios

Implementación: Procedimiento para comparar precios entre vendors

```

DELIMITER $$
CREATE PROCEDURE ComparacionPrecio(
    IN ticker_comparacion varchar(10),
    IN fecha_inicio date,
    IN fecha_fin date,
    IN fecha_consulta date)
BEGIN
    — Implementaci n completa del procedimiento de comparaci n
    — Crea tabla temporal, calcula diferencias y determina estado
END $$
DELIMITER ;

```

14.5. RQF 24 - Validaci3n Manual

Implementaci3n: Procedimiento para validaci3n manual por usuario

```

DELIMITER $$
CREATE PROCEDURE ValidacionManual(
    IN ticker_val varchar(5),
    IN fecha_val date,
    IN region_val varchar(25))
BEGIN
    — Implementaci n con funciones auxiliares para validaci n manual
END $$
DELIMITER ;

```

14.6. RQF 25 - Conteo de Validaciones

Implementaci3n: Procedimiento para contar validaciones por estado

```

DELIMITER $$
CREATE PROCEDURE ConteoValidaciones()
BEGIN
    SELECT estadoValidacion as "Estado", count(*)
    FROM resultado
    GROUP BY estadoValidacion;
END $$
DELIMITER ;

```

14.7. RQF 26 - Trigger de Validación Automática

Implementación: Trigger para validación automática al insertar activos

```
DELIMITER //
```

```
CREATE TRIGGER guardar_resultado_validacion
```

```
AFTER INSERT ON activo
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    — Asigna estado basado en el precio del activo
```

```
END;
```

```
//
```

```
DELIMITER ;
```

14.8. RQF 27 - Consulta Resultados No Validados

Implementación: Consulta de resultados no validados o semi-validados

```
SELECT r.tickerResultado , a.precioActivo , a.fechaNeg ,
```

```
        r.estadoValidacion , r.observacionResultado
```

```
FROM resultado r
```

```
JOIN activo a ON r.idActivoFK = a.idActivo
```

```
WHERE a.fechaNeg BETWEEN '2025-01-01' AND '2025-12-31'
```

```
AND r.estadoValidacion IN ( 'Unvalidated' , 'Semi-Validated' );
```

14.9. RQF 28 - Consulta Inconsistencias por Vendor

Implementación: Consulta de inconsistencias para un vendor específico

```
SELECT v.nombreVendor , r.tickerResultado , a.precioActivo ,
```

```
        r.estadoValidacion , r.observacionResultado
```

```
FROM resultado r
```

```
JOIN activo a ON r.idActivoFK = a.idActivo
```

```
JOIN detalleRecepcion dr ON a.idActivo = dr.idActivoFK
```

```
JOIN vendor v ON dr.idVendorFK = v.idVendor
```

```
WHERE v.nombreVendor = 'ICE'
```

```
AND r.estadoValidacion IN ( 'Unvalidated' , 'Semi-Validated' );
```

14.10. RQF 29 - Filtro Avanzado de Activos

Implementación: Procedimiento para filtro avanzado de activos

```

DELIMITER $$
CREATE PROCEDURE filterActive(
    IN p_nombreVendor VARCHAR(50),
    IN p_claseActivo VARCHAR(50),
    IN p_fechaInicio TIMESTAMP,
    IN p_fechaFin TIMESTAMP)
BEGIN
    SELECT a.*, v.nombreVendor
    FROM activo a
    JOIN detalleRecepcion dr ON a.idActivo = dr.idActivoFk
    JOIN vendor v ON dr.idVendorFK = v.idVendor
    WHERE (p_nombreVendor IS NULL OR v.nombreVendor = p_nombreVendor)
        AND (p_claseActivo IS NULL OR a.claseActivo = p_claseActivo)
        AND (p_fechaInicio IS NULL OR a.timestampRecepcion >= p_fechaInicio)
        AND (p_fechaFin IS NULL OR a.timestampRecepcion <= p_fechaFin);
END$$
DELIMITER ;

```

14.11. RQF 30 - Vista de Monitoreo de Inconsistencias

Implementación: Vista para monitoreo de inconsistencias en precios

```

CREATE VIEW vistaMonitoreoInconsistencias AS
SELECT
    r.tickerResultado as 'Ticker',
    r.estadoValidacion as 'Estado',
    a.fechaNeg as 'Fecha_Negociacion',
    — Campos adicionales para m tricas de inconsistencia
FROM resultado r
JOIN activo a ON r.tickerResultado = a.tickerUniversal
JOIN detalleRecepcion dr ON a.idActivo = dr.idActivoFK
JOIN vendor v ON dr.idVendorFK = v.idVendor
WHERE r.estadoValidacion IN ( 'Semi-Validated', 'Unvalidated' )
GROUP BY r.tickerResultado, r.estadoValidacion, a.fechaNeg;

```

14.12. RQF 32 - Trigger de Reversión Manual

Implementación: Trigger para prevenir reversión de validaciones no manuales

```

DELIMITER $$

```

```

CREATE TRIGGER ReversionManual
BEFORE DELETE ON resultado
FOR EACH ROW
BEGIN
    IF old.estadoValidacion NOT LIKE "Us%" THEN
        SIGNAL SQLSTATE "45000"
        SET MESSAGE_TEXT = "El resultado no es un User Validation";
    END IF;
END $$
DELIMITER ;

```

14.13. RQF 33 - Vista Final de Datos

Implementación: Vista consolidada de datos del sistema

```

CREATE VIEW vista_final_dv AS
SELECT
    v.nombreVendor as "Vendor",
    a.tickerUniversal as "Ticker",
    a.claseActivo as "Tipo-de-activo",
    a.precioActivo as "Precio",
    a.timestampRecepcion as "Timestamp",
    r.estadoValidacion as "Estado-de-validaci n"
FROM detalleRecepcion dr
JOIN vendor v ON dr.idVendorFK = v.idVendor
JOIN activo a ON dr.idActivoFK = a.idActivo
LEFT JOIN resultado r ON a.tickerUniversal = r.tickerResultado
ORDER BY a.timestampRecepcion ASC;

```

14.14. RQF 34 - Estadísticas de Validación

Implementación: Consulta de estadísticas de validación por tipo

```

SELECT
    estadoValidacion as "Tipo-de-validaci n",
    count(*) as "Cantidad-de-validaciones",
    group_concat(distinct tickerResultado order by tickerResultado separator
    as "Tickers-asociados"
FROM resultado
GROUP BY estadoValidacion;

```

14.15. RQF 35 - Consulta Activos por Vendor

Implementación: Procedimiento para consultar activos por vendor

DELIMITER \$\$

```
CREATE PROCEDURE ConsultarActivosPorVendor(IN p_nombreVendor VARCHAR(50))
BEGIN
    SELECT a.idActivo , a.tickerUniversal , a.precioActivo , a.timestampRecepcion ,
           a.divisaActivo , a.regionActivo , a.claseActivo , a.fechaNeg ,
           v.nombreVendor , r.estadoValidacion , r.observacionResultado
    FROM activo a
    JOIN detalleRecepcion dr ON a.idActivo = dr.idActivoFK
    JOIN vendor v ON dr.idVendorFK = v.idVendor
    LEFT JOIN resultado r ON a.idActivo = r.idActivoFK
    WHERE v.nombreVendor = p_nombreVendor
    ORDER BY a.timestampRecepcion DESC;
END$$
DELIMITER ;
```

14.16. RQF 36 - Consulta Activos por Tipo

Implementación: Procedimiento para consultar activos por tipo

DELIMITER \$\$

```
CREATE PROCEDURE ConsultarActivosPorTipo(IN p_claseActivo VARCHAR(50))
BEGIN
    SELECT a.idActivo , a.tickerUniversal , a.precioActivo , a.timestampRecepcion ,
           a.divisaActivo , a.regionActivo , a.claseActivo , a.fechaNeg ,
           v.nombreVendor , r.estadoValidacion
    FROM activo a
    JOIN detalleRecepcion dr ON a.idActivo = dr.idActivoFK
    JOIN vendor v ON dr.idVendorFK = v.idVendor
    LEFT JOIN resultado r ON a.idActivo = r.idActivoFK
    WHERE a.claseActivo = p_claseActivo
    ORDER BY a.timestampRecepcion DESC;
END$$
DELIMITER ;
```

14.17. RQF 37 - Consulta Activos por Timestamp

Implementación: Procedimiento para consultar activos por rango de tiempo

```

DELIMITER $$
CREATE PROCEDURE ConsultarActivosPorTimestamp(
    IN p_fechaInicio TIMESTAMP,
    IN p_fechaFin TIMESTAMP)
BEGIN
    SELECT a.idActivo , a.tickerUniversal , a.precioActivo , a.timestampRecepcion ,
           a.divisaActivo , a.regionActivo , a.claseActivo , a.fechaNeg ,
           v.nombreVendor , r.estadoValidacion , r.observacionResultado
    FROM activo a
    JOIN detalleRecepcion dr ON a.idActivo = dr.idActivoFK
    JOIN vendor v ON dr.idVendorFK = v.idVendor
    LEFT JOIN resultado r ON a.idActivo = r.idActivoFK
    WHERE a.timestampRecepcion BETWEEN p_fechaInicio AND p_fechaFin
    ORDER BY a.timestampRecepcion DESC;
END$$
DELIMITER ;

```

14.18. RQF 39 - Modificación de Validación

Implementación: Procedimiento para modificar validaciones con auditoría

```

DELIMITER $$
CREATE PROCEDURE ModificarValidacion(
    IN p_idResultado INT,
    IN p_idUsuario INT,
    IN p_nuevoEstado VARCHAR(20) ,
    IN p_nuevaObservacion VARCHAR(255) ,
    IN p_motivo VARCHAR(255))
BEGIN
    — Implementación con registro de auditoría y actualización
END$$
DELIMITER ;

```

15. Requisitos Adicionales Identificados

15.1. Trigger de Error de Precio

Implementación: Trigger para detectar errores de precio inconsistentes

```

DELIMITER $$

```

```

CREATE TRIGGER registroErrorPrecio
AFTER INSERT ON activo
FOR EACH ROW
BEGIN
    DECLARE conteo INT;
    SELECT COUNT(DISTINCT precioActivo) INTO conteo
    FROM activo
    WHERE tickerUniversal = new.tickerUniversal;

    IF conteo > 2 THEN
        INSERT INTO errorValidacion (idActivoFK, tipoError, descripcionError)
        VALUES (new.idActivo, 'Precio-inconsistente',
            'Más-de-2-precios-distintos-registrados-para-este-ticker. ');
    END IF;
END $$
DELIMITER ;

```

15.2. Vista de Validación por Ticker

Implementación: Vista para cálculo automático de estado de validación

```

CREATE OR REPLACE VIEW vw_validacion_por_ticker AS
SELECT
    a.tickerUniversal AS ticker,
    COUNT(DISTINCT a.precioActivo) AS distintos_precios,
    COUNT(DISTINCT d.idVendorFK) AS proveedores_registrados,
    CASE
        WHEN COUNT(DISTINCT a.precioActivo) = 1 THEN 'Validated'
        WHEN COUNT(DISTINCT a.precioActivo) = 2 THEN 'Semi-Validated'
        ELSE 'Unvalidated'
    END AS estado_calculado
FROM activo a
JOIN detalleRecepcion d ON a.idActivo = d.idActivoFK
GROUP BY a.tickerUniversal;

```

16. Arquitectura del Proyecto

El proyecto sigue una arquitectura MVC (Modelo-Vista-Controlador) con los siguientes componentes:

- **Modelo:** Esquema MongoDB y métodos estáticos
- **Controlador:** Manejo de requests y responses HTTP
- **Rutas:** Definición de endpoints API
- **Middlewares:** Manejo de errores y validaciones
- **Servidor:** Configuración Express y conexión a base de datos

17. Configuración del Servidor

17.1. server.js

Configuración principal de la aplicación Express:

```
// Dependencias requeridas
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const connectDB = require('./config/database');
const errorHandler = require('./middlewares/errorHandler');
const solactiveRoutes = require('./routes/solactiveRoutes');
```

Características de configuración:

- Puerto: `process.env.PORT` o 3000 por defecto
- Middlewares incluidos: CORS, JSON parsing, URL encoding
- Logger en desarrollo para tracking de requests
- Ruta raíz con documentación de endpoints
- Manejo de errores no capturados y excepciones

Endpoints documentados en ruta raíz:

- **GET /** - Documentación automática de la API
- Incluye todos los endpoints disponibles y parámetros de query

18. Base de Datos

18.1. database.js

Configuración de conexión a MongoDB:

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB Connected: ${conn.connection.host}');
  } catch (error) {
    console.error('Error connecting to MongoDB:', error.message);
    process.exit(1);
  }
};
```

Características:

- Conexión asíncrona a MongoDB usando Mongoose URI de conexión desde variable de entorno MONGO_URI
- Manejo de errores de conexión
- Terminación del proceso en caso de error de conexión

19. Middlewares

19.1. errorHandler.js

Middleware global para manejo de errores:

Tipos de errores manejados:

- **CastError:** IDs de MongoDB inválidos

```
if (err.name === 'CastError') {
  return res.status(400).json({
    success: false,
    message: 'Recurso no encontrado',
    error: 'ID no válido'
  });
}
```

- **ValidationError:** Errores de validación de Mongoose

```
if (err.name === 'ValidationError') {
  const messages = Object.values(err.errors).map(error => error.message);
  return res.status(400).json({
    success: false,
    message: 'Error de validación',
    errors: messages
  });
}
```

- **Duplicate Key Error:** Violación de índices únicos (código 11000)

```
if (err.code === 11000) {
  const field = Object.keys(err.keyPattern)[0];
  return res.status(400).json({
    success: false,
    message: 'Valor duplicado',
    error: `El campo "${field}" ya existe en la base de datos`
  });
}
```

- **Generic Server Error:** Errores no especificados

```
res.status(err.statusCode || 500).json({
  success: false,
  message: err.message || 'Server Error',
  ...(process.env.NODE_ENV === 'development' && { stack: err.stack })
});
```

Características adicionales:

- Logging detallado en consola con formato visual
- Stack trace solo en entorno de desarrollo
- Respuestas JSON estandarizadas

19.2. validateId.js

Middleware para validación de ObjectId de MongoDB:

```
const mongoose = require('mongoose');
```

```
const validateId = (req, res, next) => {
  const id = req.params.id;
  if (!mongoose.Types.ObjectId.isValid(id)) {
    return res.status(400).json({
      success: false,
      message: 'Invalid ID',
      error: `"${id}" Is not a valid MongoDB ObjectId`
    });
  }
  next();
};
```

Funcionalidad:

- Valida que los parámetros `id` sean `ObjectId` válidos
- Retorna error 400 si el ID no es válido
- Continúa al siguiente middleware si la validación es exitosa

20. Modelo de Datos

20.1. Esquema Solactive

El esquema define la estructura de los documentos en la colección MongoDB:

```
const solactiveSchema = new mongoose.Schema({
  ticker: {type: String, required: true, trim: true, minlength: 1, maxlength: 50},
  ICEPrice: {type: mongoose.Schema.Types.Double, required: false},
  ICECurrency: {type: String, required: false, trim: true, minlength: 1, maxlength: 50},
  ICERecTimestamp: {type: Date, required: false},
  FactSetPrice: {type: mongoose.Schema.Types.Double, required: false},
  FactSetCurrency: {type: String, required: false, trim: true, minlength: 1, maxlength: 50},
  FactSetLateClose: {type: mongoose.Schema.Types.Double, required: false},
  FactSetLateCurrency: {type: String, required: false, trim: true, minlength: 1, maxlength: 50},
  EDIPrice: {type: mongoose.Schema.Types.Double, required: false},
  EDICurrency: {type: String, required: false, trim: true, minlength: 1, maxlength: 50},
  negDate: {type: Date, required: true},
  region: {type: String, required: true, trim: true, maxlength: 50},
  assetClass: {type: String, required: true, trim: true, maxlength: 50},
  status: {type: String, required: false, trim: true, maxlength: 25},
  user: {type: String, required: false, trim: true, maxlength: 50},
```

```

    division: {type: String, required: false, trim: true, maxlength: 50},
    validationDate: {type: Date, required: false},
    error: {
      errorType: {type: String, required: false},
      errorDate: {type: Date, required: false},
      relatedAsset: {type: String, required: false}
    }
  })

```

21. Métodos del Modelo

21.1. NewSec

Propósito: Crear un nuevo valor en la base de datos.

Parámetros:

- ticker (requerido)
- ICEPrice (opcional)
- ICECurrency (opcional)
- ICERecTimestamp (opcional)
- FactSetPrice (opcional)
- FactSetCurrency (opcional)
- FactSetLateClose (opcional)
- FactSetLateCurrency (opcional)
- EDIPrice (opcional)
- EDICurrency (opcional)
- negDate (requerido)
- region (requerido)
- assetClass (requerido)

Respuesta: Documento del valor creado.

21.2. UpdateSec

Propósito: Actualizar campos de un valor existente.

Parámetros requeridos: ticker, negDate

Parámetros opcionales: ICEPrice, ICECurrency, ICERecTimestamp, FactSetPrice, FactSetCurrency, FactSetLateClose, FactSetLateCurrency, EDIPrice, EDICurrency, region, assetClass

Respuesta: Documento actualizado.

21.3. DeleteFields

Propósito: Eliminar campos específicos de un valor.

Parámetros requeridos: ticker, negDate

Parámetros opcionales: ICEPrice, ICECurrency, ICERecTimestamp, FactSetPrice, FactSetCurrency, FactSetLateClose, FactSetLateCurrency, EDIPrice, EDICurrency, region, assetClass

Respuesta: Documento con campos eliminados.

21.4. Validation

Propósito: Validación automática de precios.

Parámetros requeridos: ticker, negDate

Lógica:

- Compara precios de ICE, EDI y FactSet
- Asigna estados: VALIDATED, SEMI-VALIDATED, UNVALIDATED
- Maneja casos con/sin FactSetLateClose

Respuesta: Resultado de la validación con estado.

21.5. UserValidation

Propósito: Validación manual por usuario.

Parámetros requeridos: ticker, negDate, user

Respuesta: Confirmación de validación con estado "USER-VALIDATION".

21.6. UserValidationReversal

Propósito: Revertir validación de usuario.

Parámetros requeridos: ticker, negDate, user, reason

Respuesta: Confirmación de reversión o mensaje de error si no aplica.

21.7. ValidationCount

Propósito: Contar valores por estado de validación.

Parámetros: Ninguno

Respuesta: Agregación de conteos por status.

21.8. ErrorConsulting

Propósito: Consultar errores por tipo y fecha.

Parámetros opcionales: errorType, startDate, endDate, errorDate

Respuesta: Documentos que coinciden con los criterios de error.

21.9. ConsultingBy

Propósito: Consulta general con múltiples filtros.

Parámetros opcionales: Todos los campos del esquema

Respuesta: Documentos que coinciden con los filtros.

21.10. ViewAllSecurities

Propósito: Crear vista de todos los valores.

Parámetros: Ninguno

Respuesta: Todos los documentos con campos proyectados.

21.11. ViewValidationCount

Propósito: Crear vista de conteos de validación.

Parámetros: Ninguno

Respuesta: Vista agregada de conteos por status.

21.12. FilteredSecuritiesView

Propósito: Crear vista filtrada de valores.

Parámetros opcionales: assetClass, region, status

Respuesta: Vista filtrada de documentos.

21.13. ErrorTypeCount

Propósito: Contar errores por tipo.

Parámetros: Ninguno

Respuesta: Agregación de conteos por tipo de error.

21.14. VersionComp

Propósito: Comparar versiones por períodos de fecha.

Parámetros requeridos: startDate, endDate1, endDate2

Respuesta: Comparación de conteos de status y errores entre dos períodos.

21.15. DeleteDocument

Propósito: Eliminar documento completo.

Parámetros requeridos: ticker, negDate, confirmation (true)

Respuesta: Confirmación de eliminación o mensaje de error.

22. Controladores

22.1. Estructura de Respuesta

Todos los controladores siguen el patrón:

```
{
  success: boolean ,
  message: string ,
  data: object/array ,
  error: string (solo en errores)
}
```

22.2. Endpoints Implementados

22.2.1. Gestión de Valores

- **POST** /api/solactive/securities - Crear nuevo valor
- **PUT** /api/solactive/securities - Actualizar valor
- **PUT** /api/solactive/securities/fields - Eliminar campos
- **DELETE** /api/solactive/securities - Eliminar documento

22.2.2. Validaciones

- **PUT** /api/solactive/validation/auto - Validación automática
- **PUT** /api/solactive/validation/user - Validación de usuario
- **PUT** /api/solactive/validation/reversal - Reversión de validación

22.2.3. Consultas

- **GET /api/solactive/securities/consult** - Consulta general
- **GET /api/solactive/securities/consult/errors** - Consulta de errores

22.2.4. Vistas

- **GET /api/solactive/views/securities** - Vista completa de valores
- **GET /api/solactive/views/securities/filtered** - Vista filtrada
- **GET /api/solactive/views/validationCount** - Vista de conteos

22.2.5. Reportes

- **GET /api/solactive/reports/validationCount** - Reporte de validaciones
- **GET /api/solactive/reports/errorTypes** - Reporte de tipos de error
- **GET /api/solactive/reports/versionComparison** - Comparación de versiones

23. Rutas

Todas las rutas están definidas en el archivo de rutas y mapean a los controladores correspondientes. La base de todas las rutas es **/api/solactive**.

24. Validaciones

Los controladores incluyen validaciones básicas para parámetros requeridos:

- **ticker** y **negDate** son requeridos en la mayoría de operaciones
- **user** es requerido para validaciones manuales
- **confirmation: true** es requerido para eliminaciones

25. Manejo de Errores

El sistema implementa un manejo de errores comprehensivo:

- Middleware global de errores con logging detallado
- Validación de **ObjectId** de MongoDB

- Manejo de errores de base de datos (validación, duplicados, cast)
- Respuestas JSON estandarizadas para errores
- Manejo de errores no capturados y excepciones no manejadas

26. Configuración y Variables de Entorno

Variables de entorno requeridas:

- MONGO_URI: URI de conexión a MongoDB
- PORT: Puerto del servidor (opcional, default: 3000)
- NODE_ENV: Entorno de ejecución (development/production)

27. Certificaciones MongoDB

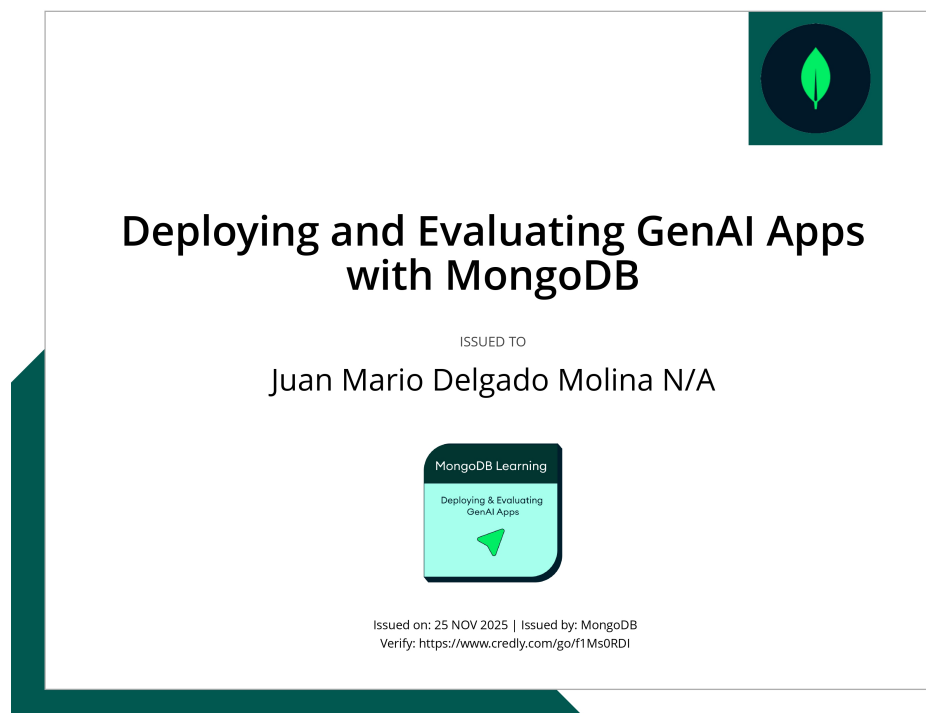


Figura 5: Certificación MongoDB Juan Mario Delgado Molina

Proof of Completion

Congratulations to
Nicolle Garaviz

For successfully completing
Deploying and Evaluating GenAI Apps Learning Badge Path

On 11-20-2025

Raghu Viswanathan
Raghu Viswanathan
VP, Education, Academia, and Documentation
MongoDB, Inc



MDBqzg0xqrsyd

Figura 6: Certificación MongoDB Nicolle Stefany Garaviz Sanchez

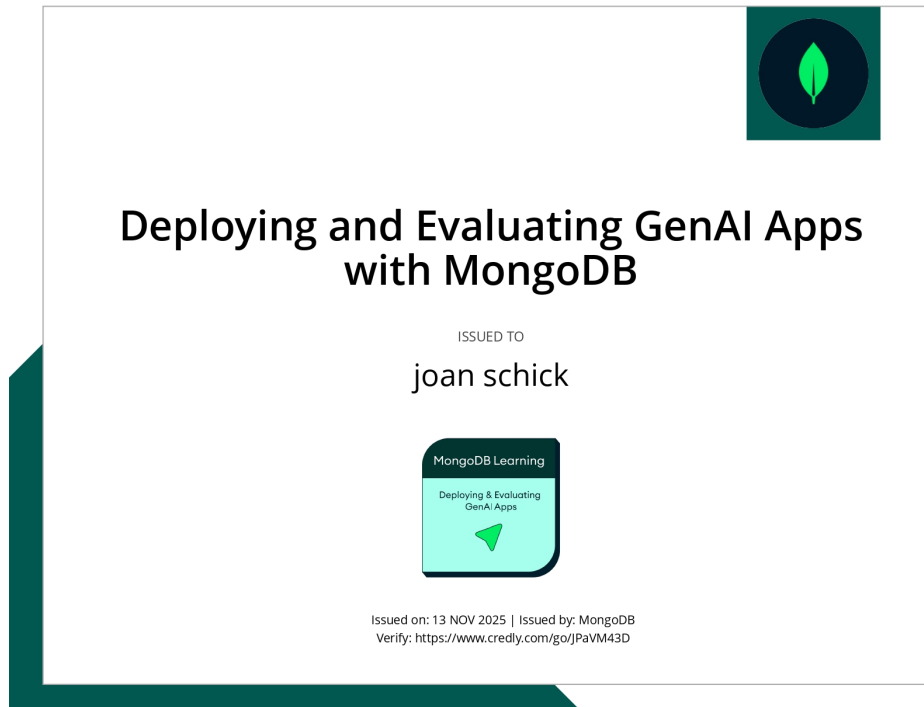


Figura 7: Certificación MongoDB Joan Sebastian Schick Reyes